

Remote Access VPN Lab Project

Raspberry Pi + WireGuard + Cisco 1941 Router + Cisco 2960 Switch

Project Overview

Project Title: Remote Access VPN Lab Using Raspberry Pi, WireGuard, Cisco 1941 Router, and Cisco 2960 Switch

Summary: Built a secure remote-access VPN into a private home lab network using a Raspberry Pi as the WireGuard VPN server, a Cisco 1941 router for routing/NAT, and a Cisco 2960 switch for lab connectivity. The project allows a laptop on an outside network to connect through a VPN tunnel and remotely manage internal lab infrastructure without exposing Cisco SSH directly to the public internet.

Objectives

- Deploy WireGuard VPN on a Raspberry Pi using PiVPN.
- Place the Raspberry Pi inside the lab network and use it as the VPN entry point.
- Configure port forwarding across both the home router and Cisco 1941 router.
- Use DuckDNS/DDNS so the VPN can connect using a domain name instead of a changing public IP.
- Configure split tunneling so only lab and VPN traffic uses the tunnel.
- Reach the Cisco router remotely through the VPN and manage it with SSH/PuTTY.
- Document troubleshooting steps, validation tests, and lessons learned.

Lab Topology

```
Remote Laptop on External Network / Phone Hotspot
|
| WireGuard VPN connection to lumi*****.org:51820
|
Home Router / Verizon Router
Port Forward: UDP 51820 -> 192.168.1.218
|
Cisco 1941 Router
WAN/Outside: 192.168.1.218
LAN/Inside: 10.10.10.1/24
Static NAT: 192.168.1.218:51820 -> 10.10.10.4:51820
|
Cisco 2960 Switch
|
Raspberry Pi WireGuard Server
Lab IP: 10.10.10.4 | VPN Tunnel IP: 10.44.190.1
|
Lab Network: 10.10.10.0/24
```

IP Addressing and Services

Component	Address / Value	Purpose
Cisco 1941 WAN Interface	192.168.1.218	Outside interface toward home router
Cisco 1941 Lab Interface	10.10.10.1/24	Default gateway for lab network
Raspberry Pi Lab IP	10.10.10.4/24	WireGuard VPN server on the lab LAN
WireGuard Server Tunnel IP	10.44.190.1	VPN-side address of Raspberry Pi
Laptop VPN Tunnel IP	10.44.190.2/32	VPN-side address assigned to laptop profile
Lab Network	10.10.10.0/24	Internal network reached after VPN connection
DuckDNS Endpoint	Lumi*****.org	Stable remote VPN hostname
WireGuard Port	UDP 51820	VPN listening port

Raspberry Pi IP a output showing eth0 and wg0

```

lumi@raspberrypi:~ $ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.10.4 netmask 255.255.255.0 broadcast 10.10.10.255
    inet6 fe80::6a12:f201:1a7:195f prefixlen 64 scopeid 0x20<link>
    ether d8:3a:dd:d8:94:2a txqueuelen 1000 (Ethernet)
    RX packets 336572 bytes 354984445 (338.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 324846 bytes 363950787 (347.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 37 bytes 4291 (4.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 37 bytes 4291 (4.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wg0: flags=209<UP,POINTOPOINT,RUNNING,NOARP> mtu 1420
    inet 10.44.190.1 netmask 255.255.255.0 destination 10.44.190.1
    inet6 fd11:5ee:bad:c0de::a2c:be01 prefixlen 64 scopeid 0x0<global>
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 1000 (UNSPEC)
    RX packets 81121 bytes 16221628 (15.4 MiB)
    RX errors 25 dropped 0 overruns 0 frame 25
    TX packets 255579 bytes 340177456 (324.4 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::943d:e60b:2673:f040 prefixlen 64 scopeid 0x20<link>
    inet6 2600:4041:40af:200:8c7c:f6ca:a013:211b prefixlen 64 scopeid 0x0<global>
    ether d8:3a:dd:d8:94:2b txqueuelen 1000 (Ethernet)
    RX packets 23177 bytes 2590592 (2.4 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3606 bytes 389480 (380.3 KiB)
    TX errors 0 dropped 7 overruns 0 carrier 0 collisions 0

lumi@raspberrypi:~ $

```

Implementation Steps

1. Prepared the Raspberry Pi on the lab LAN

- Connected the Raspberry Pi to the Cisco 2960 switch inside the 10.10.10.0/24 lab network.
- Confirmed the Raspberry Pi used 10.10.10.4 and the Cisco router lab interface 10.10.10.1 as the gateway.

2. Installed PiVPN and WireGuard

- Installed PiVPN on the Raspberry Pi and selected WireGuard as the VPN protocol.
- Created a separate WireGuard tunnel network, 10.44.190.0/24, to keep VPN addressing separate from the lab LAN.

3. Created a laptop WireGuard profile

- Generated a laptop VPN profile using pivpn add.
- Confirmed the laptop received a dedicated tunnel address and imported the .conf file into the Windows WireGuard client.

4. Configured Cisco static NAT for WireGuard

- Identified Cisco G0/0 as the WAN/outside interface and G0/1 as the lab/inside interface.
- Configured a static UDP NAT rule to forward WireGuard traffic from the Cisco WAN IP to the Raspberry Pi.

5. Configured home router port forwarding

- Created a UDP 51820 port-forwarding rule on the home router pointing to the Cisco WAN IP, 192.168.1.218.
- Confirmed the path: Internet -> Home Router -> Cisco Router -> Raspberry Pi.

6. Added DuckDNS dynamic DNS

- Created the DuckDNS domain lumi*****.org.
- Created a DuckDNS update script on the Raspberry Pi and scheduled it to run every five minutes with cron.
- Updated PiVPN so new client profiles use lumi*****.org instead of the raw public IP.

7. Tested VPN from an external network

- Connected the laptop to a phone hotspot to test from outside the home network.
- Verified WireGuard handshake and successfully pinged 10.44.190.1, 10.10.10.4, and 10.10.10.1.

8. Enabled remote Cisco SSH management

- Configured local SSH authentication on the Cisco router.
- Used PuTTY to connect to 10.10.10.1 through the VPN after resolving SSH compatibility issues with the older Cisco platform.

Key Configuration Snippets

Cisco 1941 Static NAT for WireGuard

```
ip nat inside source static udp 10.10.10.4 51820 192.168.1.218 51820 extendable
```

Pelumi Johnson

WireGuard Client Split-Tunnel Route

```
AllowedIPs = 10.10.10.0/24, 10.44.190.0/24
```

DuckDNS Cron Job

```
* /5 * * * * ~/duckdns/duck.sh >/dev/null 2>&1
```

Cisco SSH Configuration Summary

```
username admin privilege 15 secret [password hidden]
ip ssh version 2
line vty 0 4
  login local
  transport input ssh
exec-timeout 10 0
```

Problems Encountered and How They Were Solved

VPN tunnel IP looked different from the lab IP

The Raspberry Pi had both a physical lab IP and a WireGuard tunnel IP. This was expected: 10.10.10.4 was the real lab IP on eth0, while 10.44.190.1 was the VPN-side address on wg0.

Client profile transfer caused formatting/key concerns

The first profile was copied manually, which created risk of formatting issues and key exposure. A fresh profile was generated and transferred locally using a temporary Python HTTP file server on the Raspberry Pi.

WireGuard sent traffic but did not complete a handshake

Cisco NAT translations showed UDP 51820 traffic reaching the router. tcpdump on the Raspberry Pi confirmed packets arrived at eth0, proving the forwarding path worked. The fix was to recreate and import a clean WireGuard profile.

Testing from the same home network gave confusing results

The laptop was initially tested from the same network it was trying to VPN into. Retesting from a phone hotspot confirmed the VPN worked from a true external network.

DuckDNS setup required careful manual entry

The DuckDNS update command was manually entered on the Raspberry Pi. After confirming duck.log returned OK, the update script was scheduled with cron and PiVPN was updated to generate profiles with the DuckDNS endpoint.

Cisco SSH failed from modern SSH clients

The Cisco 1941 offered older SSH algorithms that modern Windows/OpenSSH clients rejected. PuTTY was used to connect successfully after the VPN proved the router was reachable.

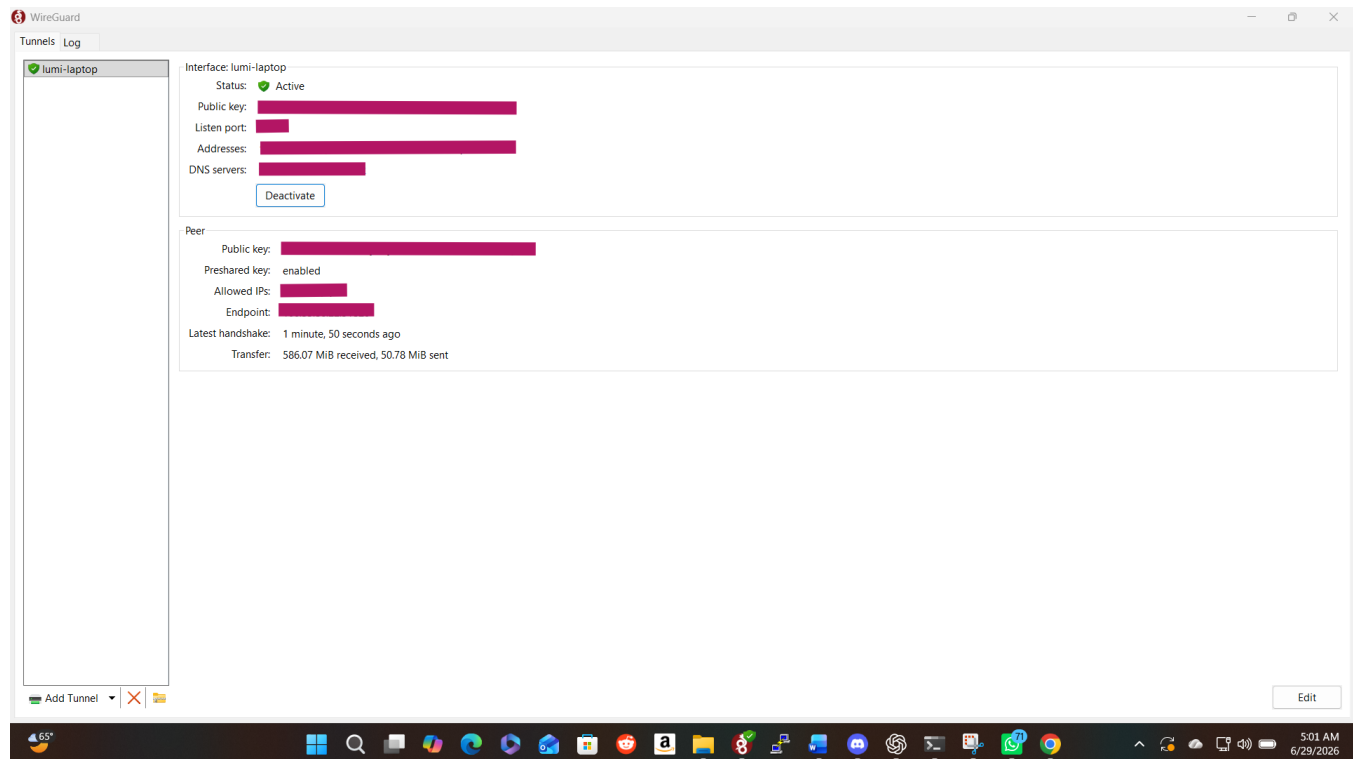
Validation Evidence

- WireGuard client displayed a successful latest handshake and traffic counters.

Pelumi Johnson

- Laptop successfully pinged the Raspberry Pi tunnel IP: 10.44.190.1.
- Laptop successfully pinged Raspberry Pi lab IP: 10.10.10.4.
- Laptop successfully pinged Cisco router lab IP: 10.10.10.1.
- Cisco NAT translations showed UDP 51820 forwarding to the Raspberry Pi.
- tcpdump confirmed UDP 51820 packets reached the Raspberry Pi.
- PuTTY successfully opened an SSH session to the Cisco router through the VPN.

WireGuard successful handshake screenshot



successful ping to 10.44.190.1, 10.10.10.4, and 10.10.10.1

```
Windows PowerShell
IPv4 Address. . . . . : 
Subnet Mask . . . . . : 
Default Gateway . . . . . : 

PS C:\Users\pelum> ping 10.44.190.1

Pinging 10.44.190.1 with 32 bytes of data:
Reply from 10.44.190.1: bytes=32 time=56ms TTL=64
Reply from 10.44.190.1: bytes=32 time=42ms TTL=64
Reply from 10.44.190.1: bytes=32 time=49ms TTL=64
Reply from 10.44.190.1: bytes=32 time=65ms TTL=64

Ping statistics for 10.44.190.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 42ms, Maximum = 65ms, Average = 53ms
PS C:\Users\pelum> ping 10.10.10.4

Pinging 10.10.10.4 with 32 bytes of data:
Reply from 10.10.10.4: bytes=32 time=37ms TTL=64
Reply from 10.10.10.4: bytes=32 time=47ms TTL=64
Reply from 10.10.10.4: bytes=32 time=49ms TTL=64
Reply from 10.10.10.4: bytes=32 time=66ms TTL=64

Ping statistics for 10.10.10.4:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 37ms, Maximum = 66ms, Average = 49ms
PS C:\Users\pelum> ping 10.10.10.1

Pinging 10.10.10.1 with 32 bytes of data:
Reply from 10.10.10.1: bytes=32 time=55ms TTL=254
Reply from 10.10.10.1: bytes=32 time=61ms TTL=254
Reply from 10.10.10.1: bytes=32 time=77ms TTL=254
Reply from 10.10.10.1: bytes=32 time=40ms TTL=254

Ping statistics for 10.10.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 40ms, Maximum = 77ms, Average = 58ms
PS C:\Users\pelum> |
```

Cisco show ip nat translations screenshot

```
COM4 - PuTTY
LAB-R1>en
LAB-R1#show ip nat translations

Pro Inside global      Inside local      Outside local      Outside global
tcp 192.168.1.219:2712 10.10.10.4:2712   10.44.190.1:4943  10.44.190.1:4943
tcp 192.168.1.219:2713 10.10.10.4:2713   10.44.190.1:4943  10.44.190.1:4943
tcp 192.168.1.219:3336 10.10.10.4:3336   10.44.190.1:5980  10.44.190.1:5980
tcp 192.168.1.219:11371 10.10.10.4:11371  10.44.190.1:32522  10.44.190.1:32522
tcp 192.168.1.219:13663 10.10.10.4:13663   10.44.190.1:59443  10.44.190.1:59443
tcp 192.168.1.219:35824 10.10.10.4:35824   10.44.190.1:100443  10.44.190.1:100443
tcp 192.168.1.219:35831 10.10.10.4:35831   10.44.190.1:234443  10.44.190.1:234443
tcp 192.168.1.219:49728 10.10.10.4:49728   10.44.190.1:93443  10.44.190.1:93443
udp 192.168.1.219:50885 10.10.10.4:50885   10.44.190.1:94443  10.44.190.1:94443
tcp 192.168.1.219:51533 10.10.10.4:51533   10.44.190.1:75443  10.44.190.1:75443
tcp 192.168.1.219:51535 10.10.10.4:51535   10.44.190.1:8883  10.44.190.1:8883
tcp 192.168.1.219:51537 10.10.10.4:51537   10.44.190.1:20443  10.44.190.1:20443
udp 192.168.1.219:51820 10.10.10.4:51820   10.44.190.1:21467  10.44.190.1:21467
udp 192.168.1.219:51820 10.10.10.4:51820   10.44.190.1:21467  10.44.190.1:21467
udp 192.168.1.219:51844 10.10.10.4:51844   10.44.190.1:215443  10.44.190.1:215443
udp 192.168.1.219:52385 10.10.10.4:52385   10.44.190.1:39443  10.44.190.1:39443
tcp 192.168.1.219:56523 10.10.10.4:56523   10.44.190.1:18.188.5228  10.44.190.1:18.188.5228
tcp 192.168.1.219:57967 10.10.10.4:57967   10.44.190.1:7.192443  10.44.190.1:7.192443
udp 192.168.1.219:59592 10.10.10.4:59592   10.44.190.1:218.100443  10.44.190.1:218.100443
udp 192.168.1.219:59606 10.10.10.4:59606   10.44.190.1:45.84443  10.44.190.1:45.84443
udp 192.168.1.219:60332 10.10.10.4:60332   10.44.190.1:81.32443  10.44.190.1:81.32443
udp 192.168.1.219:62691 10.10.10.4:62691   10.44.190.1:53  10.44.190.1:53
udp 192.168.1.219:62691 10.10.10.4:62691   10.44.190.1:12.11253  10.44.190.1:12.11253
udp 192.168.1.219:64155 10.10.10.4:64155   10.44.190.1:3  10.44.190.1:3
udp 192.168.1.219:64155 10.10.10.4:64155   10.44.190.1:12.11253  10.44.190.1:12.11253
udp 192.168.1.219:49323 10.10.10.6:49323   10.44.190.1:3  10.44.190.1:3
udp 192.168.1.219:49506 10.10.10.6:49506   10.44.190.1:3443  10.44.190.1:3443
udp 192.168.1.219:49563 10.10.10.6:49563   10.44.190.1:36443  10.44.190.1:36443
udp 192.168.1.219:49593 10.10.10.6:49593   10.44.190.1:3443  10.44.190.1:3443
udp 192.168.1.219:49595 10.10.10.6:49595   10.44.190.1:43  10.44.190.1:43
tcp 192.168.1.219:50197 10.10.10.6:50197   10.44.190.1:188443  10.44.190.1:188443
tcp 192.168.1.219:50290 10.10.10.6:50290   10.44.190.1:139443  10.44.190.1:139443
tcp 192.168.1.219:50419 10.10.10.6:50419   10.44.190.1:136443  10.44.190.1:136443
udp 192.168.1.219:50495 10.10.10.6:50495   10.44.190.1:89443  10.44.190.1:89443
udp 192.168.1.219:50678 10.10.10.6:50678   10.44.190.1:3  10.44.190.1:3
udp 192.168.1.219:50839 10.10.10.6:50839   10.44.190.1:443  10.44.190.1:443
udp 192.168.1.219:50896 10.10.10.6:50896   10.44.190.1:11.3443  10.44.190.1:11.3443
tcp 192.168.1.219:51003 10.10.10.6:51003   10.44.190.1:3.19443  10.44.190.1:3.19443
tcp 192.168.1.219:51463 10.10.10.6:51463   10.44.190.1:31.136443  10.44.190.1:31.136443
tcp 192.168.1.219:51496 10.10.10.6:51496   10.44.190.1:13.66443  10.44.190.1:13.66443
udp 192.168.1.219:51552 10.10.10.6:51552   10.44.190.1:43  10.44.190.1:43
udp 192.168.1.219:51553 10.10.10.6:51553   10.44.190.1:3  10.44.190.1:3
udp 192.168.1.219:51665 10.10.10.6:51665   10.44.190.1:9  10.44.190.1:9
udp 192.168.1.219:51675 10.10.10.6:51675   10.44.190.1:3443  10.44.190.1:3443
udp 192.168.1.219:51863 10.10.10.6:51863   10.44.190.1:3443  10.44.190.1:3443
udp 192.168.1.219:51881 10.10.10.6:51881   10.44.190.1:43  10.44.190.1:43
tcp 192.168.1.219:52298 10.10.10.6:52298   10.44.190.1:43  10.44.190.1:43
tcp 192.168.1.219:52499 10.10.10.6:52499   10.44.190.1:74.40443  10.44.190.1:74.40443
udp 192.168.1.219:52790 10.10.10.6:52790   10.44.190.1:43  10.44.190.1:43
udp 192.168.1.219:53030 10.10.10.6:53030   10.44.190.1:3443  10.44.190.1:3443
udp 192.168.1.219:53269 10.10.10.6:53269   10.44.190.1:443  10.44.190.1:443
udp 192.168.1.219:53833 10.10.10.6:53833   10.44.190.1:46443  10.44.190.1:46443
tcp 192.168.1.219:53852 10.10.10.6:53852   10.44.190.1:47443  10.44.190.1:47443
tcp 192.168.1.219:53855 10.10.10.6:53855   10.44.190.1:88443  10.44.190.1:88443
tcp 192.168.1.219:53994 10.10.10.6:53994   10.44.190.1:11443  10.44.190.1:11443
tcp 192.168.1.219:53995 10.10.10.6:53995   10.44.190.1:53443  10.44.190.1:53443
tcp 192.168.1.219:53996 10.10.10.6:53996   10.44.190.1:33:80  10.44.190.1:33:80
```

tcpdump UDP 51820 capture screenshot

```

lumi@raspberrypi:~$ sudo tcpdump -ni eth0 udp port 51820
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
03:22:23.068586 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:23.346355 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 144
03:22:23.378635 IP 10.10.10.4.51820 > [REDACTED].21467: UDP, length 144
03:22:23.460846 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 80
03:22:23.968456 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:23.968537 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:23.968537 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 96
03:22:23.968537 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:23.968537 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:23.968588 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:23.968588 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:24.373165 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 96
03:22:24.373252 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.373253 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:24.373253 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.373253 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:24.373253 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.733125 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.733125 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:24.780960 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.787372 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:24.787462 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.787463 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:24.787463 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:25.476965 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:25.966947 IP 10.10.10.4.51820 > [REDACTED].21467: UDP, length 80
03:22:25.966947 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:26.035801 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:26.146937 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 96
03:22:26.787756 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:26.787839 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:26.787839 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:26.787839 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:26.787840 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:26.787840 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:26.787840 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:26.787949 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 96
03:22:26.787949 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:27.228892 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:27.228892 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:27.228892 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 128
03:22:27.228892 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112
03:22:27.228892 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 208
03:22:27.228892 IP 10.10.10.4.51820 > [REDACTED].21467: UDP, length 112
03:22:27.228892 IP [REDACTED].21467 > 10.10.10.4.51820: UDP, length 112

```

Home Switch Telnet/SSH configuration screenshot

```

COM3 - PuTTY
switchport mode access
spanning-tree portfast
!
interface FastEthernet0/47
switchport mode access
spanning-tree portfast
!
interface FastEthernet0/48
switchport mode access
spanning-tree portfast
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/2
!
interface Vlan1
ip address 10.10.10.2 255.255.255.0
!
ip default-gateway 10.10.10.1
ip http server
ip http secure-server
!
line con 0
line vty 0 4
login local
transport input ssh
line vty 5 15
login local
transport input ssh
!
end

LAB-SW1#
LAB-SW1#
LAB-SW1#
LAB-SW1#
LAB-SW1#conf t
Enter configuration commands, one per line. End with CNTL/Z.
LAB-SW1(config)#line vty 0 4
LAB-SW1(config-line)#transport telnet
^
% Invalid input detected at '^' marker.

LAB-SW1(config-line)#transport input telnet
LAB-SW1(config-line)#transport input telnet ssh

```

Home Router Telnet/SSH configuration screenshot

```
COM4 - PuTTY
!
interface GigabitEthernet0/0
 description CONNECTED-TO-HOME-ROUTER
 ip address dhcp
 ip nat outside
 ip virtual-reassembly
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 description CONNECTED-TO-2960-SWITCH
 ip address 10.10.10.1 255.255.255.0
 ip nat inside
 ip virtual-reassembly
 duplex auto
 speed auto
!
ip forward-protocol nd
!
no ip http server
no ip http secure-server
!
ip nat inside source list 1 interface GigabitEthernet0/0 overload
ip nat inside source static udp 10.10.10.4 51820 192.168.1.218 51820 extendable
ip route 0.0.0.0 0.0.0.0 192.168.1.1
!
ip access-list standard VPN_MGMT_ONLY
 permit 10.44.190.0 0.0.0.255
 permit 10.10.10.0 0.0.0.255
 deny any
!
access-list 1 permit 10.10.10.0 0.0.0.255
!
!
!
control-plane
!
!
line con 0
line aux 0
line vty 0 4
 login local
 transport input telnet ssh
!
scheduler allocate 20000 1000
end
LAB-R1#
LAB-R1#
```

PuTTY SSH session to Cisco router screenshot

```
lumi@raspberrypi: ~
Minimum = 37ms, Maximum = 66ms, Average = 49ms
PS C:\Users\pelum> ping 10.10.10.1

Pinging 10.10.10.1 with 32 bytes of data:
Reply from 10.10.10.1: bytes=32 time=55ms TTL=254
Reply from 10.10.10.1: bytes=32 time=61ms TTL=254
Reply from 10.10.10.1: bytes=32 time=77ms TTL=254
Reply from 10.10.10.1: bytes=32 time=40ms TTL=254

Ping statistics for 10.10.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 40ms, Maximum = 77ms, Average = 58ms
PS C:\Users\pelum> ssh lumi@10.10.10.4
lumi@10.10.10.4's password:
Linux raspberrypi 6.18.34+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.18.34-1+rpt1 (2026-06-09) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 29 00:34:37 2026 from 10.44.190.2
lumi@raspberrypi:~$ telnet 10.10.10.1
Trying 10.10.10.1...
Connected to 10.10.10.1.
Escape character is '^['.

User Access Verification

Username: admin
Password:
LAB-R1#show ip int br
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0  192.168.1.218  YES DHCP    up          up
GigabitEthernet0/1  10.10.10.1     YES NVRAM    up          up
NVI0           unassigned     YES unset   administratively down down
LAB-R1#show running-config
Building configuration...
```

Security Controls Implemented

- Exposed only the WireGuard UDP port needed for VPN access; Cisco SSH was not opened directly to the internet.
- Used VPN-first access for internal device management, so Cisco SSH is reached only after the tunnel is established.
- Used split tunneling so only lab and VPN traffic routes through WireGuard.
- Stopped the temporary Python HTTP file server after downloading the client configuration file.
- Regenerated/cleaned up client profiles after the first manual transfer and kept the working DDNS-based profile.
- Configured SSH local authentication on the Cisco router and saved the running configuration.

Final Outcome

The project was completed successfully. A laptop connected to an external network can now connect to the home lab through WireGuard using the DuckDNS endpoint, reach the Raspberry Pi VPN server, access the 10.10.10.0/24 lab network, and remotely manage the Cisco router through SSH/PuTTY.

Final Working Path:

```
Laptop on External Network
-> WireGuard VPN
-> Lumi*****.org
-> Home Router Port Forward
-> Cisco 1941 Static NAT
-> Raspberry Pi WireGuard Server
-> 10.10.10.0/24 Lab Network
-> Cisco Router SSH Access
```

Skills Demonstrated

WireGuard VPN deployment	Raspberry Pi server configuration
Cisco NAT and port forwarding	Dynamic DNS with DuckDNS
VPN split tunneling	Remote SSH management
Cisco IOS CLI	Linux troubleshooting with tcpdump and ss
Windows WireGuard client setup	Layered network troubleshooting